An Experimental Study of Reinforcement Learning Algorithms in Sokoban

Saurab Mishra[1*]

[1*]Indian Institute of Science Education and Research Thiruvananthapuram, India.

Corresponding author(s). E-mail(s): saurab23@iisertvm.ac.in;

Abstract

Sokoban is a canonical PSPACE-complete planning problem characterized by combinatorial state-space explosion, irreversible actions, and complex spatial constraints, making it a challenging benchmark for decision-making algorithms. While classical search methods such as $\boldsymbol{A^*}$ guarantee optimal solutions under admissible heuristics, their reliance on exhaustive exploration and large frontier storage limits scalability to larger instances.

In this work, we reformulate Sokoban as a Markov Decision Process (MDP) and investigate reinforcement learning (RL) as a scalable alternative to explicit state-space search. We consider an $\mathbf{8} \times \mathbf{8}$ environment with increased structural complexity and implement three representative RL algorithms: Q-Learning (off-policy tabular), SARSA (on-policy tabular), and Deep Q-Networks (DQN) with function approximation.

We conduct a systematic empirical evaluation using convergence speed, policy quality, and memory efficiency as primary metrics. Our results reveal clear algorithmic trade-offs: Q-Learning achieves rapid convergence but exhibits high variance due to aggressive updates; SARSA learns more conservative and stable policies that better avoid deadlocks; and DQN demonstrates improved scalability through generalization, albeit with approximation-induced instability.

These findings highlight the fundamental transition from search-based optimal planning to experience-driven policy learning, emphasizing how algorithmic design choices, particularly policy type and representation—impact performance in structured planning environments. This study provides insights into the suitability of RL methods for combinatorial domains traditionally dominated by search-based approaches.

[1]Code repository: https://github.com/SaurabMishra12/comparative-study-of-search-and-rl-in-sokoban

Keywords: Sokoban, Reinforcement Learning, Q-Learning, SARSA, Deep Q-Network, Markov Decision Process

1 Introduction

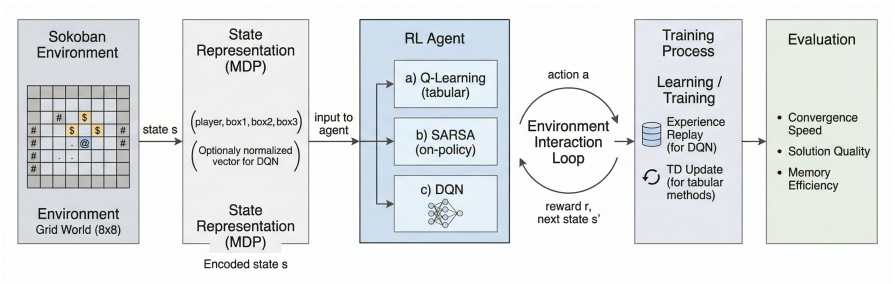

Fig. 1 Overview of the proposed reinforcement learning framework for Sokoban. The environment is modeled as an MDP, where the agent (Q-Learning, SARSA, or DQN) interacts through actions and receives rewards. The learned policies are evaluated based on convergence speed, solution quality, and memory efficiency.

Sokoban is a canonical benchmark in artificial intelligence, known for its combinatorial state-space explosion, irreversible actions, and tightly coupled spatial constraints [1] . These properties make even small instances computationally challenging and position Sokoban as a representative testbed for evaluating decision-making and planning algorithms. In prior work, we formulated Sokoban as a deterministic state-space search problem and analyzed classical algorithms such as Breadth-First Search (BFS) and A^* search [2]. While A^* achieves an effective balance between optimality and efficiency under admissible heuristics, its dependence on explicit frontier expansion and memory-intensive data structures limits its applicability to larger and more complex instances.

This limitation motivates a shift from explicit search to learning-based approaches. Reinforcement Learning (RL) provides a fundamentally different paradigm [3] in which an agent learns a policy through interaction with the environment rather than exhaustively enumerating state transitions. By optimizing long-term cumulative reward, RL enables scalable decision-making in large or partially explored state spaces, where classical search becomes infeasible.

In this work, we reformulate Sokoban as a Markov Decision Process (MDP) and investigate the effectiveness of RL algorithms in this domain. We consider three representative methods spanning key design axes in RL: Q-Learning (off-policy, tabular), SARSA (on-policy, tabular), and Deep Q-Networks (DQN) with function approximation. These methods allow us to systematically study the impact of policy type (on-policy vs. off-policy) and representation (tabular vs. parametric) on learning dynamics.

Our goal is to analyze how these algorithms differ in terms of convergence behavior, policy quality, and memory efficiency in an expanded Sokoban environment. Through controlled experiments, we aim to understand whether RL can overcome the scalability limitations of classical search while maintaining effective decision-making in structured planning problems.

2 Problem Formulation and Environment

2.1 State and Action Space

We formulate Sokoban as a Markov Decision Process (MDP) defined over a structured grid environment. Unlike the classical search formulation, which represents a state as a tuple (p, B) of player and box positions, the reinforcement learning setting requires an explicit, fixed-dimensional representation suitable for value estimation and function approximation.

The environment consists of an 8×8 grid with 3 movable boxes, 3 goal locations, and 28 static walls, leaving 36 valid navigable cells. This configuration significantly increases the combinatorial complexity compared to smaller benchmark instances.

State Space: Each state is encoded as a tuple of integer coordinates:

$$s = (\text{player}_r, \text{player}_c, \text{box1}_r, \text{box1}_c, \text{box2}_r, \text{box2}_c, \text{box3}_r, \text{box3}_c).$$

This representation captures the full configuration of the environment. For tabular methods, this tuple is used directly as a discrete state identifier. For DQN, the representation is extended to include goal coordinates, resulting in a 14-dimensional input vector, normalized to $[0, 1]$ to stabilize training.

Action Space: The agent operates in a discrete action space:

$$A = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}.$$

Each action deterministically updates the state subject to Sokoban constraints: movements into walls are disallowed, and box pushes are valid only if the destination cell is unoccupied.

2.2 Reward Structure

Sokoban naturally presents a sparse-reward setting, where meaningful feedback is obtained only upon task completion. To facilitate efficient learning, we introduce a carefully designed reward shaping mechanism that provides intermediate guidance while preserving the original task objective.

The reward function is defined as follows:

- **+100**: Assigned upon reaching the terminal state where all boxes are correctly placed on goal locations.
- **-1**: Step penalty applied at each timestep to encourage shorter solution trajectories and discourage unnecessary exploration.

- **-5**: Penalty for invalid actions, including collisions with walls or infeasible box pushes, discouraging illegal transitions.
- **+2**: Shaping reward provided when an action reduces the aggregate Manhattan distance between boxes and their nearest goals. This serves as a weak heuristic signal to guide exploration without overwhelming the primary objective.

This reward design balances exploration and exploitation by providing dense feedback while maintaining alignment with the true task objective. In particular, the shaping term is intentionally kept small to avoid biasing the agent toward locally optimal but globally suboptimal behaviors.

3 Algorithms Implemented

We consider three representative reinforcement learning algorithms [3] that span key design dimensions: policy type (on-policy vs. off-policy) and representation (tabular vs. function approximation). All methods are implemented within a unified experimental framework to ensure fair comparison.

3.1 Q-Learning (Off-Policy Tabular TD Control)

Q-Learning is an off-policy temporal-difference control algorithm that directly approximates the optimal action-value function, independent of the behavior policy. Its update rule is given by:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]. \tag{1}$$

By bootstrapping from the maximum estimated future value, Q-Learning aggressively propagates rewards backward through the state space. This enables rapid convergence toward optimal policies but may introduce instability during early training due to overestimation bias.

In our implementation, Q-Learning employs an ϵ-greedy exploration strategy with a fast decay schedule ($\epsilon : 1.0 \rightarrow 0.01$), encouraging early exploration followed by rapid exploitation. This configuration prioritizes convergence speed, often at the cost of higher variance in learning dynamics.

3.2 SARSA (On-Policy Tabular TD Control)

SARSA is an on-policy temporal-difference method that evaluates and improves the policy actually being executed. Its update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right], \tag{2}$$

where a' is the next action sampled from the current policy.

Unlike Q-Learning, SARSA incorporates the effects of exploratory actions into its updates, resulting in more conservative value estimates. This property is particularly beneficial in environments such as Sokoban, where exploratory moves can lead to irreversible deadlocks.

We pair SARSA with a slower ϵ-decay schedule ($\epsilon : 1.0 \to 0.05$), maintaining a higher degree of exploration throughout training. This leads to smoother convergence and safer policies, albeit with slower learning compared to Q-Learning.

3.3 Deep Q-Network (DQN)

To address the scalability limitations of tabular methods, we implement a Deep Q-Network (DQN) [4] that approximates the action-value function using a neural network. This allows generalization across similar states without explicitly storing all state-action pairs.

The network architecture consists of fully connected layers:

$$\text{Input}(14) \to \text{FC}(64) \to \text{ReLU} \to \text{FC}(64) \to \text{ReLU} \to \text{FC}(32) \to \text{ReLU} \to \text{FC}(4).$$

Training is performed by minimizing the temporal-difference loss:

$$\mathcal{L}(\theta) = \left(Q_\theta(s,a) - \left[r + \gamma \max_{a'} Q_{\bar{\theta}}(s',a')\right]\right)^2, \tag{3}$$

where $\bar{\theta}$ denotes the parameters of a target network.

To stabilize training, DQN incorporates two key mechanisms:

- **Experience Replay:** Transitions are stored in a replay buffer (capacity 10,000) and sampled uniformly to break temporal correlations and improve sample efficiency.
- **Target Network:** A periodically updated target network ($\bar{\theta}$, synchronized every 300 steps) provides stable targets for learning.

While DQN significantly improves scalability through function approximation, it introduces additional challenges such as approximation error, sensitivity to hyperparameters, and potential training instability.

4 Experimental Setup

We evaluate all algorithms within a controlled experimental framework to ensure a fair and consistent comparison. Each method is trained over multiple episodes, with a maximum horizon of 300 steps per episode. Episodes terminate either upon reaching the goal state (all boxes placed correctly) or when the step limit is exceeded.

To systematically compare learning behavior, we focus on three primary evaluation criteria: (i) convergence speed, measured by the number of episodes required to achieve stable performance, (ii) solution quality, assessed via cumulative reward and trajectory efficiency, and (iii) memory efficiency, reflecting the storage requirements of tabular versus function approximation methods.

All algorithms employ an ϵ-greedy exploration strategy, with decay schedules tailored to their respective learning dynamics. Tabular methods are trained for 10,000 episodes to ensure sufficient state-space coverage, while DQN is trained for 3,000 episodes due to its ability to generalize across states via function approximation.

Table 1 summarizes the key hyperparameters used across all methods. The learning rates and exploration schedules are chosen to balance stability and convergence

speed, with more conservative settings for SARSA and more aggressive updates for Q-Learning. For DQN, a smaller learning rate is used to stabilize gradient-based optimization.

Table 1 Hyperparameter settings across the evaluated RL algorithms.

Hyperparameter	**Q-Learning**	**SARSA**	**DQN**
Learning Rate	0.2	0.1	5×10^{-4}
Discount Factor (γ)	0.99	0.99	0.99
ϵ Decay Rate	0.9995	0.9998	0.998
Final ϵ	0.01	0.05	0.02
Training Episodes	10,000	10,000	3,000

5 Results and Analysis

This section presents a comparative evaluation of the three reinforcement learning algorithms along three key dimensions: convergence behavior, policy quality, and memory efficiency. The observed results are consistent with theoretical expectations regarding policy evaluation strategies and representation choices.

5.1 Convergence Speed and Solution Quality

Q-Learning: Q-Learning demonstrates the fastest convergence among the evaluated methods. By bootstrapping from the greedy maximum action, it rapidly propagates reward signals backward through the state space, resulting in efficient discovery of near-optimal policies. Empirically, this is reflected in a tight reward distribution and shorter solution trajectories. However, this aggressive update mechanism introduces high variance during early training, leading to unstable learning curves before convergence.

SARSA: SARSA exhibits slower convergence due to its on-policy nature, which incorporates the effects of exploratory actions into value updates. While this slows down reward propagation, it results in more stable and conservative learning dynamics. In the Sokoban setting, this translates to safer policies that avoid irreversible deadlocks, as the agent explicitly accounts for the risk introduced by continued exploration. Consequently, SARSA achieves more consistent performance at the cost of longer training time.

DQN: DQN achieves moderate convergence speed, benefiting from its ability to generalize across similar states via function approximation. This reduces the need for exhaustive state visitation. However, the approximation introduces instability in training, often resulting in a bimodal performance profile: the agent either converges to a near-optimal solution or fails to solve the task. This variability reflects the inherent challenges of stabilizing deep reinforcement learning in structured, constraint-heavy environments.

5.2 Memory and Sample Efficiency

Tabular Methods (Q-Learning and SARSA): Tabular approaches exhibit memory requirements that scale linearly with the number of visited state-action pairs. In the 8×8 environment, both methods expand their Q-tables to over 50,000 entries, with SARSA requiring slightly more due to prolonged exploration. While effective in small domains, this growth quickly becomes prohibitive as the state space increases, limiting scalability.

Function Approximation (DQN): In contrast, DQN maintains a fixed memory footprint determined by network parameters (approximately 5,000 parameters in our implementation), independent of the number of visited states. This provides a significant scalability advantage in larger environments. However, this efficiency comes at the cost of increased sample complexity and sensitivity to hyperparameters, as the model must learn generalized representations from limited experience.

The radar plot further highlights the trade-offs between aggressive learning (Q-Learning), safety (SARSA), and generalization (DQN), providing a consolidated summary of algorithmic behavior across multiple evaluation metrics (see Fig. 2).

5.3 Impact of Reward Shaping

Reward shaping plays a critical role in guiding exploration within the sparse-reward Sokoban environment. The introduction of a mild heuristic reward (+2) significantly accelerates learning by providing intermediate feedback aligned with the task objective.

We observe that increasing the shaping magnitude (+5) leads to a collapse in behavioral diversity, causing all algorithms to converge to similar policies. This indicates that excessive shaping can dominate the learning signal, effectively masking the intrinsic differences between algorithms. By carefully calibrating the shaping reward, we preserve meaningful distinctions in learning dynamics—highlighting Q-Learning's aggressive optimization, SARSA's risk-aware behavior, and DQN's generalization capability.

6 Conclusion

In this work, we investigated the transition from classical search-based planning to reinforcement learning in the Sokoban domain. By modeling the environment as a Markov Decision Process and evaluating Q-Learning, SARSA, and DQN under a unified framework, we analyzed the trade-offs between policy learning strategies and representation methods.

The results demonstrate that off-policy tabular methods such as Q-Learning are highly effective in small, discrete environments, achieving rapid convergence and strong solution quality. On-policy methods like SARSA provide improved robustness by explicitly accounting for exploration, making them better suited for environments with irreversible or risky transitions.

While DQN introduces approximation-related instability in smaller settings, it fundamentally addresses the scalability limitations of classical search and tabular RL by

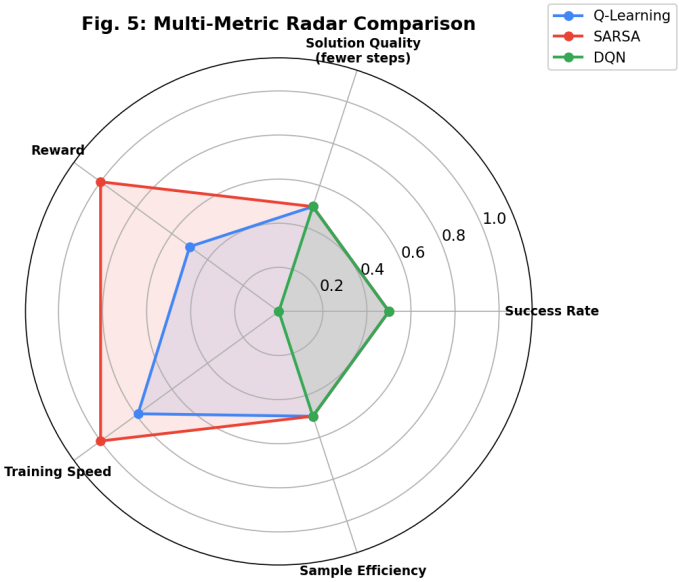

Fig. 2 Multi-metric comparison of Q-Learning, SARSA, and DQN across key performance dimensions: success rate, solution quality, reward, training speed, and sample efficiency. The radar plot highlights trade-offs between fast convergence, policy safety, and scalability.

maintaining a fixed memory footprint. This positions function approximation as a critical component for extending RL methods to larger and more complex planning problems.

Overall, the study highlights that the choice of learning paradigm must balance convergence speed, stability, and scalability. Future work may explore more advanced architectures and algorithms, such as Double DQN or policy-gradient methods, to further improve stability and performance in structured environments like Sokoban.

References

[1] Culberson, J.C.: Sokoban is pspace-complete. In: Proceedings of the International Conference on Fun with Algorithms, pp. 65–76 (1997)

[2] Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach, 3rd edn. Prentice Hall, ??? (2010)

[3] Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, ??? (2018)

[4] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015) https://doi.org/10.1038/nature14236

Declarations

Funding: Not applicable.

Conflict of interest: The author declares no competing interests.

Ethics approval and consent to participate: Not applicable.

Consent for publication: Not applicable.

Data availability: Available from the author upon reasonable request.

Code availability: Available for reproduciblity purposes.

Author contribution: Saurab Mishra conceptualized the study, implemented the algorithms, performed the experiments, and wrote the manuscript.

GenAI tools: I used GenAI tools to help prepare the materials for this study; however, I personally cross-checked and verified all content.

[1] Code repository: https://github.com/SaurabMishra12/comparative-study-of-search-and-rl-in-sokoban